diag180: Counting polyominoes with 180° rotational symmetry by a transfer matrix along diagonals

Günter Rote Claude

November 11, 2026

Abstract

This program counts the fixed polyominoes with n cells that are mapped to themselves by a rotation by 180°, for all n up to a given bound N. It builds one half of the polyomino row by row along the *diagonals* of the square grid and keeps, for every intermediate boundary configuration, only the number of ways to reach it. Configurations that cannot be completed with at most N cells are discarded with the help of a lower bound on the number of missing cells. For small n we have checked that this bound is exact. The numbers are one of the ingredients that are needed to extend the count of *free* polyominoes (OEIS A000105) beyond $n = 59$. The program is written in CWEB [8].

1 Introduction

A *polyomino* is a finite set of cells of the square grid that is connected through common edges. *Fixed* polyominoes are counted up to translation only; *free* polyominoes are counted up to translation, rotation and reflection. By Burnside's lemma, the number of free polyominoes with n cells is

$$\frac{1}{8}\sum_{g\in D_4} \mathrm{fix}_g(n),$$

where $\mathrm{fix}_g(n)$ is the number of fixed polyominoes with n cells that are invariant under the symmetry g of the square, up to translation. For the identity this is the number of all fixed polyominoes (OEIS A001168 [9]), which is known up to $n = 70$ by the work of Barequet and Ben-Shachar [1, 2]. (The value for $n = 67$ printed in [1] is too large by 2; it is corrected in [2].) The other seven terms count symmetric polyominoes. The number of free polyominoes is currently known up to $n = 59$ (Shirakawa [11]), and in that computation the polyominoes with a 180° rotational symmetry were the bottleneck: they were enumerated one by one.

This program computes $\mathrm{fix}_\rho(n)$ for the rotation ρ by 180° with a transfer-matrix method, in the spirit of Jensen's algorithm [4, 5] and of Knuth's programs POLYNUM and POLYSLAVE [6, 7], but with three special features:

1. We do not build the polyomino P but its "half", the quotient of P under ρ. The connectivity information in the state has to keep track of how each component is related to its own mirror image.
2. The half is built along the diagonals $x + y =$ const of the grid, as in the work of Barequet and Ben-Shachar on fixed polyominoes [1]. Connections between neighbouring components then use *disjoint sets of columns*, which makes it possible to compute the cost of the cheapest completion of a state almost exactly.
3. States that cannot lead to a polyomino with at most N cells are pruned, using this cost. In our experiments the growth of the number of states is about 1.25 per cell, compared to roughly 1.43 for counting all fixed polyominoes with the same technique.

The program is called in one of the following ways:

```
diag180 [options] N           all cases; prints n and fix(n) for n = 1..N
diag180 [options] N c d vhi   one case (see below); prints n, #lt, #eq
```

The options are described in the section on the command line.

¶ The overall structure of the program is conventional.

2 ⟨ Include files 3 ⟩
⟨ Type definitions 14 ⟩
⟨ Global variables 8 ⟩
⟨ Subroutines 9 ⟩
⟨ The main program 84 ⟩

3 ¶ ⟨ Include files 3 ⟩ ≡
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <time.h>
```
This code is used in chunk 2.

¶ Some format declarations for the typesetting of this document.

1.1 Symmetry classes and cases

The rotation ρ of a polyomino P that is invariant under it has a centre, and after a translation we may assume that

$$\rho(x, y) = (c - x,\ d - y) \qquad \text{with } c, d \in \{0, 1\}.$$

The centre is the point $(c/2, d/2)$. There are three types of centres:

- $(c, d) = (0, 0)$: the centre of a cell. The centre cell is the only cell that is its own image. Hence n is odd if the centre cell belongs to P, and even if it does not (as for a ring around an empty centre). (Type R180C in Shirakawa's notation [11].)
- $(c, d) = (1, 0)$ or $(0, 1)$: the midpoint of a vertical or horizontal edge. Here n is even. The two cases are mirror images of each other under $x \leftrightarrow y$ and give the same counts; we compute only $(1, 0)$ and count it twice. (Type R180M.)
- $(c, d) = (1, 1)$: a vertex of the grid; n is even. (Type R180V.)

These sets are disjoint, and $\mathrm{fix}_\rho(n)$ is the sum of the counts over all centres. For odd n, only the centre type C contributes, and in our experiments the run for $N = 2k + 1$ needed no larger tables than the run for $N = 2k$.

1.2 Diagonal coordinates

We use the coordinates

$$u = x + y, \qquad v = x - y.$$

A cell (u, v) exists whenever $u + v$ is even, and its four neighbours are $(u \pm 1, v \pm 1)$. The set of cells with a fixed u is a *row* (a diagonal of the original grid); the set of cells with fixed v is a *column*. A neighbour of a cell is always in an adjacent row *and* in an adjacent column. In the pictures, u increases upwards and v to the right, so cells appear as diamonds.

In these coordinates, $\rho(u, v) = (s - u,\ t - v)$ with $s = c + d$ and $t = c - d$. Let $v_{\max}$ be the largest v-coordinate of P. By symmetry the smallest is $t - v_{\max}$. A *case* is a triple $(c, d, v_{\max})$: we count the symmetric polyominoes that live in the strip $t - v_{\max} \le v \le v_{\max}$ of

$$L = 2v_{\max} - t + 1$$

columns and touch its two sides. (Touching one side implies touching the other.) Inside the program the columns are numbered by their *position* $i = v - v_{\min}$, $0 \le i < L$, and the image of position i is position $L - 1 - i$.

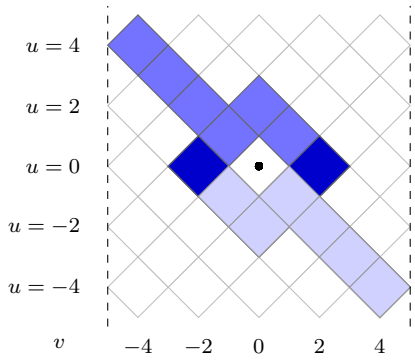


Figure 1: A polyomino with 14 cells and centre $(c, d) = (0, 0)$ in diagonal coordinates: a 3×3 ring around the empty centre (dot) with two arms. The strip has $L = 9$ columns, $v = -4\,..\,4$. The middle row $u = 0$ contains two cells (dark); the cells of H above it are drawn in blue, their images in light blue.

¶ **Wide and tall polyominoes.** The reflection $y \to -y$ exchanges u and v. It maps polyominoes with a 180° symmetry to polyominoes with a 180° symmetry, and it exchanges the extent in the u-direction (the number of rows, *uspan*) with the extent in the v-direction (*vspan* $= L$). It is therefore sufficient to generate the polyominoes with vspan $\le$ uspan:

$$\mathrm{fix}_\rho(n) = \sum_{\text{cases}} \big(2 \cdot \#\{\text{vspan} < \text{uspan}\} + \#\{\text{vspan} = \text{uspan}\}\big).$$

This has two advantages: the strips are narrow ($L \le N/2$ or so matters most), and the requirement that P must still grow to a height of at least L rows strengthens the pruning.

¶ The geometry of the current case is kept in global variables.

8 `#define MAXL 160 /* maximum number of columns of the strip */`

⟨ Global variables 8 ⟩ ≡
```
static int NMAX; /* the bound N on the number of cells */
static int L, vlo, T, S, E; /* L, v_min, t, s, and s mod 2 */
static int NW; /* length of a vector of counters, NMAX + 1 */
```
See also chunks 13, 15, 26, 33, 53, 69, 71, and 83

This code is used in chunk 2.

1.3 The half polyomino

Let H be the set of cells of P with $u > s/2$, together with the cells of the *middle row* $u = s/2$ if s is even. Then $P = H \cup \rho(H)$. The program constructs H row by row, with increasing u, starting at the middle, and within each row from left to right, deciding for each cell whether it belongs to H or not.

Every cell of H outside the middle row stands for two cells of P and contributes 2 to the size n. We call the contribution to n the *weight*. If s is even, the middle row is mapped to itself. We do not build it cell by cell; we start with all possible symmetric subsets of the middle row. If s is odd, there is no middle row, but the first row $u_0 = (s + 1)/2$ of H is adjacent to its own image, the row $u_0 - 1$. We call it the *twin row*; it needs a special treatment of adjacencies.

After k rows beyond the middle have been built, the polyomino extends over $2k + 1$ rows if s is even and over $2k$ rows if s is odd.

9 ⟨ Subroutines 9 ⟩ ≡
```
static int uspan(int k)
{
  return 2 * k + 1 - E;
}
```
See also chunks 16, 17, 18, 19, 20, 21, 23, 24, 29, 30, 31, 34, 35, 37, 38, 39, 40, 41, 42, 43, 44, 46, 54, 55, 56, 57, 58, 59, 60, 61, 67, 68, 70, 72, and 73

This code is used in chunk 2.

2 The state

At any time during the construction, some cells have been decided and the others are still open. The boundary between them is a zigzag line: if the cells of row u have been decided up to position i, the most recently decided cell in each column is in row u for the positions $\le i$ and in row $u-1$ for the positions $> i$ (always only for the columns of the right parity). These cells form the *frontier*. The place where the frontier steps down from row u to row $u-1$ is called the *kink*. A frontier cell is dropped from the frontier as soon as both of its upper neighbours have been decided, since nothing can be attached to it any more. (See Figure 2.)

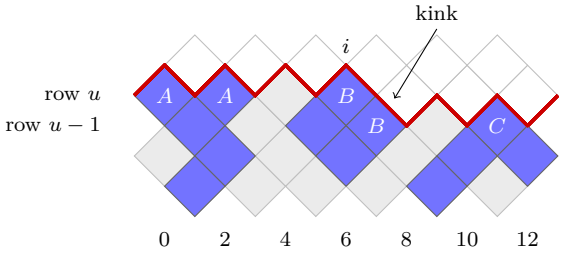


Figure 2: The frontier (thick line) after position $i = 6$ of row u has been decided. Blue cells belong to H, grey cells are decided and empty, white cells are undecided. The frontier consists of the cells of row u at the positions $0, 2, 4, 6$ and the cells of row $u-1$ at the positions $7, 9, 11$. Five of them are occupied; they form three classes A, B, C with four runs. The frontier steps down at the kink, right of position i.

¶ **Classes and partners.** Consider the part of P that is known so far: the decided cells of H and their images. Its connected components that contain a frontier cell are the *classes*. A *run* is a maximal block of consecutive positions that hold frontier cells. Since the cells of a row occupy only every other position, and the cells in between have already left the frontier, most runs consist of a single cell; only at the kink, the positions i and $i+1$ can both hold a frontier cell, and these two cells are neighbours. The frontier cells of a class form one or more runs. (In Figure 2, the two cells of class A are two runs, and class B has one run of two cells.) Since the components are separated by the frontier line from the undecided region, two classes cannot *cross*: the pattern $A \dots B \dots A \dots B$ does not occur.

The image $\rho(X)$ of a class X is again a component of the known part. There are three possibilities:

- $\rho(X) = X$. We call X a *self* class. Such a component already connects H to its image.
- $\rho(X) = Y$ is another class. Then X and Y are *partners*.
- $\rho(X)$ has no cell on the frontier. We say that the partner of X is a *ghost*.

When two classes X and Y are joined by a new cell, their images are joined as well, by the image of that cell. This is why the partner relation must be part of the state: if X and Y happen to be partners, the result is a self class; otherwise the new class $X \cup Y$ has the partner $\rho(X) \cup \rho(Y)$.

The polyomino P is connected if and only if, at the end, there is a single class and it is a self class.

¶ In addition, the state contains one bit that tells whether a side of the strip has already been *touched*, i.e., whether the leftmost or rightmost column contains a cell.

For each state we store how many different sets of decided cells lead to it, separately for each weight (number of cells of P so far).

2.1 Byte keys

In its unpacked form, a state is a *byte key* of $\texttt{KL} \leftarrow L + \texttt{MAXC} + 1$ bytes:

- $key[i]$, $0 \le i < L$: the class label of the frontier cell at position i, or 0 if there is none. Labels are numbered $1, 2, \dots$ in the order of their first appearance from left to right, which makes the key canonical.
- $key[L + j - 1]$, for each class j: its partner, which is another class label, or `PSELF`, or `PGHOST`.
- $key[\texttt{KL} - 1]$: the *touched* bit.

A frontier contains at most $\lceil L/2 \rceil + 1$ cells, hence at most `MAXC` classes.

```
13  #define PSELF  ^FF      /* the class is its own image */
    #define PGHOST  ^FE     /* the image of the class has left the frontier */
    ⟨ Global variables 8 ⟩ +≡
      static int MAXC;      /* maximum number of classes in a state */
      static int KL;        /* length of a byte key */
```

2.2 The working state

For performing a transition, a byte key is expanded into a structure with a union-find forest over the classes. In addition to the nodes $1\,..\,\texttt{MAXC}+1$ for the classes, there is a node $\texttt{GH}(j)$ for the ghost partner of each class j, because ghosts can be merged, too.

```
14  #define MAXN  2 * (MAXL + 4)      /* bound on the number of union-find nodes */
    #define GH(i)  (GOFF + (i))       /* the ghost of class i */
    #define ISGHOST(a)  ((a) > GOFF)
    ⟨ Type definitions 14 ⟩ ≡
      typedef struct {
        int f[MAXL];
        int partner[MAXL];      /* for classes: -1 self, -2 ghost, else class */
        int parent[MAXN];
        int next;       /* next free class label */
      } work_t;
```

See also chunks 22, 27, 32, and 36

This code is used in chunk 2.

```
15  ¶  ⟨ Global variables 8 ⟩ +≡
      static int GOFF;      /* offset of the ghost nodes, MAXC + 1 */
```

¶ The usual *find* operation with path halving.

```
16  ⟨ Subroutines 9 ⟩ +≡
      static inline int find(work_t *w, int a)
      {
        while (w→parent[a] ≠ a) {
          w→parent[a] ← w→parent[w→parent[a]];
          a ← w→parent[a];
        }
        return a;
      }
```

¶ The function *rho* returns the (root of the) image of the class or ghost a.

```
17  ⟨ Subroutines 9 ⟩ +≡
      static inline int rho(work_t *w, int a)
      {
        a ← find(w, a);
        if (ISGHOST(a)) return find(w, a − GOFF);
        int p ← w→partner[a];
        if (p ≡ −1) return a;
        if (p ≡ −2) return find(w, GH(a));
        return find(w, p);
      }
```

¶ When two sets are linked, a class node is preferred as the root over a ghost node. Thus the root of a set is a ghost only if the set consists entirely of ghosts.

18 ⟨ Subroutines 9 ⟩ +≡

```
static inline void link_(work_t *w, int a, int b)
{
  a ← find(w, a);
  b ← find(w, b);
  if (a ≡ b) return;
  if (ISGHOST(a)) {
    int x ← a;
    a ← b;
    b ← x;
  }
  w→parent[b] ← a;
}
```

¶ Joining two classes means joining their images as well. If b is (in the same set as) the image of a, the two unions are the same, and the result is a self class: its image, found through the *partner* link or the ghost node, is in the same set.

19 ⟨ Subroutines 9 ⟩ +≡

```
static inline void unite(work_t *w, int a, int b)
{
  int ra ← rho(w, a), rb ← rho(w, b);
  link_(w, a, b);
  link_(w, ra, rb);
}
```

¶ Expanding a byte key into the working state.

20 ⟨ Subroutines 9 ⟩ +≡

```
static void decode(work_t *w, const uint8_t *key)
{
  int m ← 0;
  for (int i ← 0; i < L; i++) {
    w→f[i] ← key[i];
    if (key[i] > m) m ← key[i];
  }
  for (int a ← 0; a < MAXN; a++) w→parent[a] ← a;
  for (int j ← 1; j ≤ MAXC; j++) {
    int q ← key[L + j − 1];
    w→partner[j] ← q ≡ PSELF ? −1 : q ≡ PGHOST ? −2 : q;
    if (q ∧ j > m) m ← j;
  }
  w→next ← m + 1;
}
```

¶ The converse operation produces the canonical byte key: it renumbers the classes from left to right and determines their partners. The function returns the number of classes.

21 ⟨ Subroutines 9 ⟩ +≡

```
static int encode(work_t *w, uint8_t *key, int tch)
{
  int map[MAXN];
  int rep[MAXL];
  int m ← 0;
```

```
  memset(map, 0, sizeof (map));
  memset(key, 0, KL);
  for (int i ← 0; i < L; i++) {
    if (¬w→f[i]) continue;
    int r ← find(w, w→f[i]);
    if (¬map[r]) {
      map[r] ← ++m;
      rep[m] ← r;
    }
    key[i] ← map[r];
  }
  for (int j ← 1; j ≤ m; j++) {
    int q ← rho(w, rep[j]);
    key[L + j − 1] ← q ≡ rep[j] ? PSELF : map[q] ? map[q] : PGHOST;
  }
  key[KL − 1] ← tch;
  return m;
}
```

2.3 Removing a cell from the frontier

When the frontier cell at position i can no longer get new neighbours, it is *retired*. If other frontier cells of the same class remain, nothing happens. Otherwise the class X disappears from the frontier, and there are several cases:

- X is a self class. Then X is a complete symmetric polyomino, and nothing can be added to it. If the frontier is now empty, we have found a polyomino (`COMPLETE`); otherwise the other classes can never be connected to X, and the state is `DEAD`.
- The partner of X is a ghost. Then X and $\rho(X)$ are two different components to which nothing can be added: `DEAD`.
- The partner of X is a class Y on the frontier. From now on, the partner of Y is a ghost.

22 ⟨ Type definitions 14 ⟩ +≡

```
enum {
  OK, DEAD, COMPLETE
};
```

23 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static int retire(work_t *w, int i)
{
  int lab ← w→f[i];
  if (¬lab) return OK;
  w→f[i] ← 0;
  int r ← find(w, lab);
  int any ← 0;
  for (int j ← 0; j < L; j++)
    if (w→f[j]) {
      any ← 1;
      if (find(w, w→f[j]) ≡ r) return OK;
    }
  int p ← rho(w, r);
  if (p ≡ r) return any ? DEAD : COMPLETE;
  if (ISGHOST(p)) return DEAD;
  w→partner[find(w, p)] ← −2;
  return OK;
}
```

2.4 A transition

The function *step* takes the byte key *sk* of a state (with touched bit *tch*) and decides the cell at position i of the current row: *occ* tells whether the cell belongs to H. The parameter *last* is true for the last cell of the row. The result is `DEAD`, `COMPLETE`, or `OK`; in the last case the new canonical key is stored in *key*, its touched bit in $*ntp$ and its number of classes in $*mp$.

A new cell gets a new class label whose partner is a ghost (its image), and it is united with the frontier cells at positions $i-1$ and $i+1$. These are its two lower neighbours: at this moment, both positions hold cells of the previous row, see Figure 2.

After the decision, the cell at position $i-1$ has no undecided neighbours any more and is retired; at the end of a row, the same holds for position $i+1$. (Retiring an empty position does no harm.)

24 ⟨ Subroutines 9 ⟩ +≡

```
static int step(work_t *w, const uint8_t *sk, int tch, int i, int occ, int last, int first_twin, uint8_t
        *key, int *ntp, int *mp)
{
  decode(w, sk);
  if (occ) {
    int nw ← w→next++;
    w→partner[nw] ← −2;
    w→f[i] ← nw;
    for (int nb ← i − 1; nb ≤ i + 1; nb += 2)
      if (nb ≥ 0 ∧ nb < L ∧ w→f[nb]) unite(w, nw, w→f[nb]);
    if (first_twin) {
      for (int jj ← L − 2 − i; jj ≤ L − i; jj += 2) {
        if (jj ≡ i) unite(w, nw, rho(w, nw));
        else if (jj ≥ 0 ∧ jj < L ∧ jj < i ∧ w→f[jj]) unite(w, nw, rho(w, w→f[jj]));
      }
    }
  }
  int status ← OK;
  if (i − 1 ≥ 0) status ← retire(w, i − 1);
  if (status ≡ OK ∧ last ∧ i + 1 < L) status ← retire(w, i + 1);
  if (status ≠ OK) return status;
  *ntp ← tch ∨ (occ ∧ (i ≡ 0 ∨ i ≡ L − 1));
  *mp ← encode(w, key, *ntp);
  return OK;
}
```

¶ **The twin row.** In the twin row $u_0 = (s+1)/2$, which exists if s is odd, the lower neighbours of the cell at position i are the cells $(u_0 - 1, v \pm 1)$. They are the images of the cells of the twin row itself at the positions $L - 1 - (i \pm 1)$, that is, $L - 2 - i$ and $L - i$. If such a position equals i, the new cell is adjacent to its own image and becomes a self class. If it is smaller than i and occupied, the new cell is united with the *image* of that cell. If it is larger than i, the adjacency will be found later, when that cell is decided. The parameter *first_twin* is true while the twin row is processed.

3 Packed keys

The byte keys are convenient but wasteful. In the tables, a state is stored as a bit string in `KW` 64-bit words:

- 3 bits per position, with one of the following codes: `C_EMPTY` for an empty position; `C_CONTINUE` if the cell continues the run of its left neighbour; and for the first cell of a run: `C_SINGLETON` if the class consists of this single run, `C_FIRST` for the first run of a class with several runs, `C_MIDDLE` for a middle run, `C_LAST` for the last run. Since classes do not cross, a stack is sufficient to recover the labels, like for a sequence of parentheses.
- 1 bit per class: 0 if the partner is a ghost, 1 otherwise.
- 1 bit: touched.

This is far from the information-theoretic minimum, but with pruning, the set of states is a sparse subset of all syntactically possible states anyway, and the keys are only a moderate part of the memory. (Ranking the states, as in [3], would give shorter keys, but it is slow.)

26 ⟨ Global variables 8 ⟩ +≡
```
static int KW;      /* length of a packed key in 64-bit words */
```

27 ¶ ⟨ Type definitions 14 ⟩ +≡
```
enum {
  C_EMPTY, C_CONTINUE, C_SINGLETON, C_FIRST, C_MIDDLE, C_LAST
};
```

¶ **Why one bit per class is enough.** Call a class *crossing* if it is a self class or has a partner on the frontier. The union of the frontier and its image under ρ is a closed curve around the known part of P (think of a disk), and a crossing class together with its partner connects the frontier with the image of the frontier across this disk. Two such connections cannot cross each other, and ρ reverses the order along the frontier. Consequently, the crossing classes, read from left to right, form one nested chain

$$A_1,\ A_2,\ \ldots,\ A_k,\ (S),\ \rho A_k,\ \ldots,\ \rho A_2,\ \rho A_1,$$

where a self class S is present in the middle if and only if the number of crossing classes is odd. In particular, there is at most one self class. Thus the j-th crossing class from the left is the partner of the j-th crossing class from the right, and it suffices to know *which* classes are crossing. With the option `-C`, the program checks for every state that unpacking the packed key gives back the original byte key. We have run this check for $N \le 30$.

¶ Bit strings are written and read with two small routines. No field has more than 3 bits.

29 ⟨ Subroutines 9 ⟩ +≡
```
static inline void putbits(uint64_t *w, int *pos, uint64_t v, int nb)
{
  int i ← *pos ≫ 6, o ← *pos & 63;
  w[i] |= v ≪ o;
  if (o + nb > 64) w[i + 1] |= v ≫ (64 − o);
  *pos += nb;
}
static inline int getbits(const uint64_t *w, int *pos, int nb)
{
  int i ← *pos ≫ 6, o ← *pos & 63;
  uint64_t v ← w[i] ≫ o;
  if (o + nb > 64) v |= w[i + 1] ≪ (64 − o);
  *pos += nb;
  return v & ((1 U ≪ nb) − 1);
}
```

¶ Packing. The first loop determines, for every class, the positions where its first and its last run start.

30 ⟨ Subroutines 9 ⟩ +≡
```
static void pack(const uint8_t *key, uint64_t *w)
{
  int first[MAXL + 2], lastrun[MAXL + 2];
  int pos ← 0, m ← 0;
  memset(w, 0, KW * 8);
  for (int i ← 0; i < L; i++)
    if (key[i] ∧ (i ≡ 0 ∨ ¬key[i − 1])) {
      int l ← key[i];
```

```
      if (l > m) {
        m ← l;
        first[l] ← i;
      }
      lastrun[l] ← i;
    }
  for (int i ← 0; i < L; i++) {
    int l ← key[i], code;
    if (¬l) code ← C_EMPTY;
    else if (i > 0 ∧ key[i − 1]) code ← C_CONTINUE;
    else if (first[l] ≡ i) code ← lastrun[l] ≡ i ? C_SINGLETON : C_FIRST;
    else code ← lastrun[l] ≡ i ? C_LAST : C_MIDDLE;
    putbits(w, &pos, code, 3);
  }
  for (int j ← 1; j ≤ m; j++) putbits(w, &pos, key[L + j − 1] ≠ PGHOST, 1);
  putbits(w, &pos, key[KL − 1], 1);
}
```

¶ Unpacking. In the second part, *stack* holds the crossing classes from left to right, and they are matched from the outside to the inside.

31 ⟨ Subroutines 9 ⟩ +≡

```
static void unpack(const uint64_t *w, uint8_t *key)
{
  int stack[MAXL + 2], sp ← 0, m ← 0, pos ← 0;
  memset(key, 0, KL);
  for (int i ← 0; i < L; i++) {
    int code ← getbits(w, &pos, 3);
    switch (code) {
    case C_EMPTY: key[i] ← 0;
      break;
    case C_CONTINUE: key[i] ← key[i − 1];
      break;
    case C_SINGLETON: key[i] ← ++m;
      break;
    case C_FIRST: key[i] ← ++m;
      stack[sp++] ← m;
      break;
    case C_MIDDLE: key[i] ← stack[sp − 1];
      break;
    case C_LAST: key[i] ← stack[−−sp];
      break;
    }
  }
  sp ← 0;      /* crossing classes, left to right */
  for (int j ← 1; j ≤ m; j++)
    if (getbits(w, &pos, 1)) stack[sp++] ← j;
    else key[L + j − 1] ← PGHOST;
  for (int a ← 0, b ← sp − 1; a ≤ b; a++, b−−) {
    key[L + stack[a] − 1] ← a ≡ b ? PSELF : stack[b];
    key[L + stack[b] − 1] ← a ≡ b ? PSELF : stack[a];
  }
  key[KL − 1] ← getbits(w, &pos, 1);
}
```

4 Counters

The numbers that we count exceed 2^{64} at $n = 66$. There are two ways to deal with this. By default, the counters are unsigned 64-bit integers, and the arithmetic is modulo 2^{64}, or modulo a number $M < 2^{64}$ given

by the option `-m`. The true values can then be recovered by the Chinese remainder theorem from two runs with different moduli, for example 2^{64} and $2^{61}-1$. (The script `tools/crt180.py` does this.) Alternatively, if the program is compiled with `-DCNT128`, the counters have 128 bits, and a single run gives the exact values. The records become twice as large, but memory is not a bottleneck.

32 ⟨ Type definitions 14 ⟩ +≡
```
#ifdef CNT128
  typedef unsigned __int128 cnt_t;      /* make CNT128=1: exact to 2^128 */
#define CNT_BITS "128"
#else
  typedef uint64_t cnt_t;
#define CNT_BITS "64"
#endif
```

33 ¶ ⟨ Global variables 8 ⟩ +≡
```
  static uint64_t MOD ← 0;      /* the modulus M; 0 means no reduction */
```

34 ¶ ⟨ Subroutines 9 ⟩ +≡
```
  static inline cnt_t addm(cnt_t a, cnt_t b)
  {
    if (¬MOD) return a + b;
    cnt_t s ← a + b;
    if (s < a ∨ s ≥ MOD) s −= MOD;
    return s;
  }
```

¶ There is no `printf` format for 128-bit numbers. The function *cs* converts a counter to a decimal string. It uses eight buffers in rotation, so that several results can be used in the same call of `printf`.

35 ⟨ Subroutines 9 ⟩ +≡
```
  static const char *cs(cnt_t x)
  {
    static char buf[8][44];
    static int r;
    char *e ← buf[r ← (r + 1) & 7] + 43, *p ← e;
    *p ← 0;
    do *−−p ← '0' + (int)(x % 10), x /= 10; while (x);
    return p;
  }
```

5 The tables

The states of one stage of the computation are stored in a hash table with open addressing (linear probing). The states are numbered $0, 1, \ldots, n-1$ in the order of their creation; *keys* contains their packed keys, and the hash table *hash* contains the state numbers plus 1.

The counters of a state form a linked list of *records*, one record per weight for which the count is nonzero. This is much more compact than a full vector of $N+1$ counters, because the weights of a state are confined to a narrow range: the pruning removes all weights that are too large, and at the peak there are only about two records per state.

36 ⟨ Type definitions 14 ⟩ +≡
```
  typedef struct {
    cnt_t c;
    uint32_t next;      /* index + 1, 0 = end */
    uint8_t wt;
  } rec_t;
  typedef struct {
    size_t n, cap, hmask;
```

```
  uint64_t *keys;
  uint32_t *head;      /* first record + 1 */
  rec_t *rec;
  size_t nrec, rcap;
  uint32_t *hash;      /* index + 1, 0 = empty */
} table_t;
```

37 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static void *xrealloc(void *p, size_t s)
{
  p ← realloc(p, s);
  if (¬p) {
    fprintf(stderr, "out␣of␣memory\n");
    exit(1);
  }
  return p;
}
```

38 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static void tab_init(table_t *t, size_t cap)
{
  t↣n ← 0;
  t↣cap ← cap;
  size_t h ← 16;
  while (h < 2 * cap) h ≪= 1;
  t↣hmask ← h − 1;
  t↣keys ← xrealloc(0, cap * KW * 8);
  t↣head ← xrealloc(0, cap * 4);
  t↣hash ← calloc(h, sizeof(uint32_t));
  t↣rcap ← 2 * cap;
  t↣nrec ← 0;
  t↣rec ← xrealloc(0, t↣rcap * sizeof(rec_t));
}
static void tab_free(table_t *t)
{
  free(t↣keys);
  free(t↣head);
  free(t↣rec);
  free(t↣hash);
}
static size_t tab_bytes(const table_t *t)
{
  return t↣n * (KW * 8 + 4) + t↣nrec * sizeof(rec_t) + (t↣hmask + 1) * 4;
}
```

¶ The hash function mixes all words of the key.

39 ⟨ Subroutines 9 ⟩ +≡

```
static inline uint64_t hkey(const uint64_t *k)
{
  uint64_t h ← #9E3779B97F4A7C15ULL;
  for (int i ← 0; i < KW; i++) {
    h ⊕= k[i];
    h *= #BF58476D1CE4E5B9ULL;
    h ⊕= h ≫ 31;
  }
  return h;
}
```

¶ When the table is full, the capacity is doubled and all states are rehashed.

40 ⟨ Subroutines 9 ⟩ +≡

```
static void tab_grow(table_t *t)
{
  size_t cap ← t↣cap * 2;
  t↣keys ← xrealloc(t↣keys, cap * KW * 8);
  t↣head ← xrealloc(t↣head, cap * 4);
  size_t h ← (t↣hmask + 1) * 2;
  free(t↣hash);
  t↣hash ← calloc(h, sizeof(uint32_t));
  if (¬t↣hash) {
    fprintf(stderr, "out␣of␣memory\n");
    exit(1);
  }
  t↣hmask ← h − 1;
  t↣cap ← cap;
  for (size_t i ← 0; i < t↣n; i++) {
    size_t p ← hkey(t↣keys + i * KW) & t↣hmask;
    while (t↣hash[p]) p ← (p + 1) & t↣hmask;
    t↣hash[p] ← i + 1;
  }
}
static void tab_clear(table_t *t)
{
  t↣n ← 0;
  t↣nrec ← 0;
  memset(t↣hash, 0, (t↣hmask + 1) * sizeof(uint32_t));
}
```

¶ The function *tab_get* returns the number of the state with a given byte key. If the state is not yet in the table, it is created, with an empty list of records.

41 ⟨ Subroutines 9 ⟩ +≡

```
static size_t tab_get(table_t *t, const uint8_t *key)
{
  uint64_t w[12];
  pack(key, w);
  if (CHECK) {
    uint8_t k2[2 * MAXL + 4];
    unpack(w, k2);
    if (memcmp(k2, key, KL)) {
      fprintf(stderr, "pack/unpack␣mismatch\n");
      exit(2);
    }
  }
  size_t p ← hkey(w) & t↣hmask;
  while (t↣hash[p]) {
    size_t i ← t↣hash[p] − 1;
    if (¬memcmp(t↣keys + i * KW, w, KW * 8)) return i;
    p ← (p + 1) & t↣hmask;
  }
  if (t↣n ≡ t↣cap) {
    tab_grow(t);
    p ← hkey(w) & t↣hmask;
    while (t↣hash[p]) p ← (p + 1) & t↣hmask;
  }
  size_t i ← t↣n++;
```

```
    t↦hash[p] ← i + 1;
    memcpy(t↦keys + i * KW, w, KW * 8);
    t↦head[i] ← 0;
    return i;
  }
```

¶ Adding c to the counter for weight wt of state si.

42 ⟨ Subroutines 9 ⟩ +≡

```
  static void tab_add(table_t *t, size_t si, int wt, cnt_t c)
  {
    for (uint32_t r ← t↦head[si]; r; r ← t↦rec[r − 1].next)
      if (t↦rec[r − 1].wt ≡ wt) {
        t↦rec[r − 1].c ← addm(t↦rec[r − 1].c, c);
        return;
      }
    if (t↦nrec ≡ t↦rcap) {
      t↦rcap *= 2;
      t↦rec ← xrealloc(t↦rec, t↦rcap * sizeof(rec_t));
    }
    rec_t *r ← &t↦rec[t↦nrec++];
    r↦c ← c;
    r↦wt ← wt;
    r↦next ← t↦head[si];
    t↦head[si] ← t↦nrec;
  }
```

¶ For processing a state, its counters are copied into a dense vector gf (“generating function”), which must be zero beforehand, and *tab_unload* clears it again without touching the whole vector.

43 ⟨ Subroutines 9 ⟩ +≡

```
  static void tab_load(const table_t *t, size_t si, cnt_t *gf)
  {
    for (uint32_t r ← t↦head[si]; r; r ← t↦rec[r − 1].next) gf[t↦rec[r − 1].wt] ← t↦rec[r − 1].c;
  }
  static void tab_unload(const table_t *t, size_t si, cnt_t *gf)
  {
    for (uint32_t r ← t↦head[si]; r; r ← t↦rec[r − 1].next) gf[t↦rec[r − 1].wt] ← 0;
  }
```

¶ Inserting into a table with pruning. The counters gf of a state, shifted by the weight add of the newly added cell, are added to the state key of table t, but only for the weights that can still lead to a polyomino with at most N cells, given that at least lb further units of weight are necessary. If nothing remains, the state is not even created.

44 ⟨ Subroutines 9 ⟩ +≡

```
  static void insert_pruned(table_t *t, const uint8_t *key, const cnt_t *gf, int add, int lb)
  {
    int lim ← NMAX − lb − add;
    int any ← 0;
    for (int wt ← 0; wt ≤ lim; wt++)
      if (gf[wt]) {
        any ← 1;
        break;
      }
    if (¬any) return;
    size_t si ← tab_get(t, key);
    for (int wt ← 0; wt ≤ lim; wt++)
      if (gf[wt]) tab_add(t, si, wt + add, gf[wt]);
  }
```

6 The lower bound

The key to the efficiency of the program is a good lower bound on the number of cells that must still be added to a state before it can become a complete polyomino that is as high as it is wide. A state whose weight plus this bound exceeds N is discarded.

All bounds in this chapter count cells of H; each such cell has weight 2.

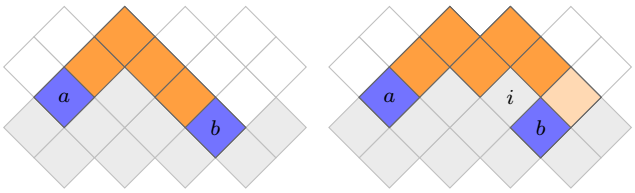
¶ First, a state in which every class has a ghost partner can never be completed: whatever is attached to a class X is attached in mirror image to $\rho(X)$, which is out of reach, so X and $\rho(X)$ can never meet. (The empty state, which occurs only in the twin row, is fine.)

```
46 ⟨ Subroutines 9 ⟩ +≡
  static int completion_possible(const uint8_t *key)
  {
    int m ← 0;
    for (int j ← 0; j < MAXC; j++) {
      int q ← key[L + j];
      if (¬q) break;
      m++;
      if (q ≠ PGHOST) return 1;
    }
    return m ≡ 0;
  }
```

6.1 Connections on the diagonal lattice

Suppose two frontier cells $a < b$ (we identify cells with their positions) in different runs have to be connected, and there are g empty positions between them. A path between them changes the column by ± 1 in every step, hence it needs at least one cell in each of the g columns strictly between a and b. Moreover, a connection between two consecutive runs and a connection between two other consecutive runs lie in disjoint sets of columns, so their costs simply add up. This is the great advantage of diagonal rows over horizontal rows: there, a single new cell can be adjacent to two frontier cells and to its successor in the row.

There is one exception, *at the kink* (Figure 3). Suppose position i has just been decided to be empty, and $b = i + 1$ is a frontier cell in row $u - 1$. The upper neighbours of b are the cell at position i of row u, which is empty, and the cell at position $i + 2$. Hence a path from the left must pass over b and come back to it from the right: it needs an additional cell in the column of b itself (and one in column $i + 2$, which we do not charge, because it might also be used for a connection to the right of b).



(a) $g = 4$ columns, 4 cells (b) at the kink: one more cell

Figure 3: Connecting two frontier cells a and b. (a) One cell per column between them. (b) The cell i left of b has been decided empty, and b is in the lower row: the path must cross the column of b above it. The last cell, drawn lighter, is not charged.

6.2 The cactus of gaps

Which gaps between consecutive runs have to be bridged? We must eventually connect all classes, where a class and its partner count as the same object (if X is connected to Y then ρX is connected to ρY). Consider the graph whose nodes are the classes, with partners identified, and with an edge for each gap between consecutive runs. Because classes do not cross and because of the nested chain structure of the crossing classes, this graph is a *cactus*: its cycles are edge-disjoint, and it can be decomposed as follows.

- *Pockets.* Between two consecutive runs of the same class X there may be classes $C_1, \ldots, C_r$ (at the outermost level inside these two runs). They can only be ghost-partnered, and they appear nowhere else. The gaps X–C_1–$\cdots$–C_r–X form a cycle. To connect everything, all gaps of the cycle except one must be bridged, and the cheapest way is to omit the largest gap. The pockets inside each C_j are independent of this.
- *Outer paths.* At the top level, the classes to the left of A_1 and to the right of ρA_1 hang on a path, and all their gaps must be bridged.
- *Rings.* The gaps from A_j to A_{j+1} on the left together with the gaps from ρA_{j+1} to ρA_j on the right form a cycle, since A_j and ρA_j are the same node. Again the largest gap can be omitted.
- *The middle.* The gaps from A_k over S (if present) to ρA_k form the innermost cycle.

The sum of all gaps minus the largest gap of each cycle is the cost for connecting the cactus.

¶ **The self term.** Connecting the quotient is not enough: in the end, there must be a *self* class. If there is none yet, some class X must be joined to its own image. Closing a ring completely does not achieve this, because the right half of a ring is just the image of its left half. Only a path across the middle, from A_k to ρA_k, creates a self class. Therefore, if there is no self class, the middle cycle must be closed completely, and its largest gap is paid as well.

¶ The function *cycle* handles one cycle with n gaps of sizes g. It returns the sum minus the maximum. The remaining parameters and the calls of *reach* and *sreach* have to do with the height, which is discussed next.

6.3 Reaching the sides and the top

The finished polyomino must touch the sides of the strip, and it must extend to row U^*, the last row for which the height reaches the width L. If no side has been touched yet, the frontier cell closest to a side needs a path of at least as many cells as there are columns between it and the side. In addition, each row above the current row up to row U^* must contain at least one cell. These requirements interact with the connections: a staircase to the side also climbs one row per cell, and the connecting paths between runs also reach some height.

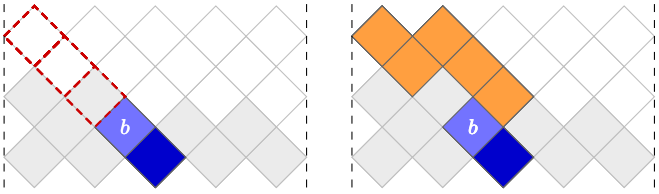
A path of c cells that connects frontier cells in rows r_a and r_b has its highest point at most in row $\lfloor (c + r_a + r_b + 1)/2 \rfloor$: it forms a "tent". The bound adds up the connection costs, determines the largest height h that the connecting paths (or the frontier itself) can reach, and charges one cell for each of the $U^* - h$ rows that are still missing.

There are choices. In a cycle, omitting the largest gap is cheapest, but bridging the largest gap and omitting the second largest one costs only the difference and may reach higher. If several gaps are tied for the largest, we may keep the higher ones for free. The bound takes the minimum over these alternatives. They are collected as *options* (extra cost, peak, score) in the arrays `OPTD`, `OPTP`, `OPTS`.

¶ When no side has been touched yet, the situation is more delicate, because the path to the side may branch off from any cell of the structure, and it helps to reach the top at the same time. From a cell in row h and column j, a staircase that reaches both a side and row U^* needs

$$\max\bigl(U^* - h,\ \min(j,\ L - 1 - j)\bigr)$$

cells (Figure 4). We call the minimum of this quantity over all cells of the structure the *score*; the cells of the structure are the frontier cells and the cells of the connecting paths at their highest possible positions. The alternative is to reach the side and the top separately, at a cost of (distance to the side) + (missing rows).



(a) the bound: 3 cells (b) the truth: 5 cells

Figure 4: The centre is in a cell, $L = 9$, $U^* = 4$. Rows 0 and 1 are complete, and in row 2 the first two cells have been decided empty. (a) The staircase bound for the cell b in row $h = 1$ and column $j = 3$ is $\max(4-1, 3) = 3$, but the first cell of the staircase is already known to be empty. (b) Every completion must use the right upper neighbour of b, and 5 cells are necessary. The lookahead at the kink finds this: it tries both possibilities for the next cell.

¶ The variables for the height computation are global, to keep the parameter lists short.

53 ⟨ Global variables 8 ⟩ +≡
```
static int CUR_U, USTAR, UO;      /* current row u, target row U*, first row u_0 */
static int HEIGHT ← 1;      /* use the height terms? (option -H turns them off) */
static int HO, SO;      /* best height and best score reached for free */
static int NOPT, OPTD[2 * MAXL + 4], OPTP[2 * MAXL + 4], OPTS[2 * MAXL + 4];      /* options */
static int NCL, CLD[MAXL], CLP[MAXL], CLS[MAXL];
    /* the middle cycle: largest gap, its peak and score */
static int DEDGE;      /* distance to the nearest side, if untouched */
static int CUR_TCH;      /* touched bit of the current state */
```

¶ The row of the frontier cell at position j: row `CUR_U` or the row below, depending on the parity.

54 ⟨ Subroutines 9 ⟩ +≡
```
static inline int frow(int j)
{
  return ((vlo + j − CUR_U) % 2 + 2) % 2 ? CUR_U − 1 : CUR_U;
}
```

55 ¶ ⟨ Subroutines 9 ⟩ +≡
```
static inline void reach(int h)
{
  if (h > HO) HO ← h;
}
static inline void sreach(int sc)
{
  if (sc < SO) SO ← sc;
}
```

¶ The cost of reaching both the top row and a side from row h and column j.

56 ⟨ Subroutines 9 ⟩ +≡
```
static inline int both_cost(int h, int j)
{
  int e ← j < L − 1 − j ? j : L − 1 − j;
  int r ← USTAR − h;
  return e > r ? e : r;
}
```

¶ The best such cost over the cells of a connecting path, each at its highest possible row. The path leads from the frontier cell a in row ra to column e, where it arrives at row $re_$ or lower.

57 ⟨ Subroutines 9 ⟩ +≡

```
static int gap_score(int a, int ra, int e, int re_)
{
  int best ← 1 ≪ 20;
  for (int j ← a + 1; j ≤ e; j++) {
    int t1 ← ra + 1 + (j − a − 1), t2 ← re_ + (e − j);
    int c ← both_cost(t1 < t2 ? t1 : t2, j);
    if (c < best) best ← c;
  }
  return best;
}
```

¶ Now we can give the function *cycle*. Besides returning the sum minus the maximum, it registers the peaks and scores of all gaps that are certainly bridged. If the largest gap *m1* is unique, bridging it instead of the second largest gap *m2* is recorded as an option with extra cost *m1* − *m2*. If the pointers are nonnull, the largest gap and its peak and score are returned (this is used for the self term).

58 ⟨ Subroutines 9 ⟩ +≡

```
static int cycle(const int *g, const int *pk, const int *sc, int n, int *m1p, int *p1p, int *s1p)
{
  int sum ← 0, m1 ← −1, n1 ← 0, m2 ← 0, p1 ← −1000, s1 ← 1 ≪ 20;
  for (int t ← 0; t < n; t++) {
    sum += g[t];
    if (g[t] > m1) {
      if (m1 ≥ 0) m2 ← m1;
      m1 ← g[t];
      n1 ← 1;
      p1 ← pk[t];
      s1 ← sc[t];
    }
    else if (g[t] ≡ m1) {
      n1 ++;
      if (pk[t] > p1) p1 ← pk[t];
      if (sc[t] < s1) s1 ← sc[t];
    }
    else if (g[t] > m2) m2 ← g[t];
  }
  for (int t ← 0; t < n; t++)
    if (n1 ≥ 2 ∨ g[t] ≠ m1) {
      reach(pk[t]);
      sreach(sc[t]);
    }
  if (n1 ≡ 1) {
    OPTD[NOPT] ← m1 − m2;
    OPTS[NOPT] ← s1;
    OPTP[NOPT++] ← p1;
  }
  if (m1p) {
    *m1p ← m1;
    *p1p ← p1;
    *s1p ← s1;
  }
  return sum − m1;
}
```

¶ The function *finish*(*h*, *s*) gives the number of cells that are still needed for reaching the top (and the side, if untouched) when the structure reaches height *h* and score *s*.

59 ⟨ Subroutines 9 ⟩ +≡
```
static inline int finish(int h, int s)
{
  int top ← USTAR − h > 0 ? USTAR − h : 0;
  if (CUR_TCH) return HEIGHT ? top : 0;
  if (¬HEIGHT) return DEDGE;
  int sep ← DEDGE + top;      /* edge and top separately */
  return s < sep ? s : sep;
}
```

¶ The function *height_extra* takes the minimum over the options. At most one option is used at a time.

60 ⟨ Subroutines 9 ⟩ +≡
```
static int height_extra(int h, int s)
{
  int ex ← finish(h, s);
  for (int o ← 0; o < NOPT ∧ ex > 0; o++) {
    int e2 ← OPTD[o] + finish(OPTP[o] > h ? OPTP[o] : h, OPTS[o] < s ? OPTS[o] : s);
    if (e2 < ex) ex ← e2;
  }
  return ex;
}
```

6.4 Putting the bound together

The parameter *kink* is the position of the frontier cell right of the kink, if the cell before it has been decided empty, and −1 otherwise.

61 ⟨ Subroutines 9 ⟩ +≡
```
static int cactus_bound(const uint8_t *key, int tch, int kink)
{
  ⟨ Find the runs of the frontier 62 ⟩
  ⟨ Initialise the height computation with the frontier cells 63 ⟩
  int cost ← 0;
  ⟨ Scan the runs from left to right and pay for the pockets 64 ⟩
  ⟨ Pay for the outer paths, the rings and the middle 65 ⟩
  ⟨ Add the self term and the terms for the side and the top 66 ⟩
  return cost;
}
```

¶ Run j extends from position $rs[j]$ to $re[j]$ and belongs to class $rl[j]$.

62 ⟨ Find the runs of the frontier 62 ⟩ ≡
```
int rs[MAXL], re[MAXL], rl[MAXL], R ← 0;
int lastrun[MAXL + 2];
for (int i ← 0; i < L; i++)
  if (key[i]) {
    if (i ≡ 0 ∨ ¬key[i − 1]) {
      rs[R] ← i;
      rl[R] ← key[i];
      R++;
    }
    re[R − 1] ← i;
    lastrun[key[i]] ← R − 1;
  }
if (¬R) return 0;
```
This code is used in chunk 61.

63 ¶ ⟨ Initialise the height computation with the frontier cells 63 ⟩ ≡

```
HO ← −1000;
SO ← 1 ≪ 20;
NOPT ← 0;
NCL ← 0;
CUR_TCH ← tch;
for (int i ← 0; i < L; i++)
  if (key[i]) {
    reach(frow(i));
    sreach(both_cost(frow(i), i));
  }
```

This code is used in chunk 61.

¶ The pockets are processed with a stack, like nested parentheses. The gaps of the open pockets are collected in a buffer (gb, with peaks pb and scores qb); sl and sb give the class of each open pocket and the start of its gaps in the buffer. Gaps at the top level are stored separately: $topgap[t]$ is the gap after the t-th top-level class $toplab[t]$. When a run of the class on top of the stack is found, the pocket is closed and paid for.

64 ⟨ Scan the runs from left to right and pay for the pockets 64 ⟩ ≡

```
    /* open pockets: class and start of its gaps in the gap buffer */
int sl[MAXL], sb[MAXL], sp ← 0;
int gb[MAXL], pb[MAXL], qb[MAXL], nb ← 0;
int toplab[MAXL], topgap[MAXL], toppk[MAXL], topsc[MAXL], nt ← 0;
for (int j ← 0; j < R; j++) {
  int X ← rl[j];
  if (j > 0) {
    int a ← re[j − 1], b ← rs[j];
    int g ← b − a − 1 + (b ≡ kink);      /* at the kink the path ends one column beyond b: g + 2 cells */
    int pk ← (g − 1 + (b ≡ kink) + frow(a) + frow(b) + 2)/2;
    int sc ← tch ? 1 ≪ 20 : gap_score(a, frow(a), b ≡ kink ? b + 1 : b − 1, frow(b) + 1);
    if (sp) {
      gb[nb] ← g;
      qb[nb] ← sc;
      pb[nb++] ← pk;
    }
    else {
      topgap[nt − 1] ← g;
      topsc[nt − 1] ← sc;
      toppk[nt − 1] ← pk;
    }
  }
  if (sp ∧ sl[sp − 1] ≡ X) {
    sp−−;
    cost += cycle(gb + sb[sp], pb + sb[sp], qb + sb[sp], nb − sb[sp], 0, 0, 0);
    nb ← sb[sp];
  }
  else if (¬sp) toplab[nt++] ← X;
  if (j < lastrun[X]) {
    sl[sp] ← X;
    sb[sp] ← nb;
    sp++;
  }
}
```

This code is used in chunk 61.

¶ At the top level, cr lists the crossing classes. Ring number i consists of the gaps between $cr[i]$ and $cr[i+1]$ and their counterparts on the right; the last "ring", $i = k − 1$, is the middle.

65 ⟨ Pay for the outer paths, the rings and the middle 65 ⟩ ≡ /* top level */

```
int cr[MAXL], m ← 0;
for (int t ← 0; t < nt; t++)
  if (key[L + toplab[t] − 1] ≠ PGHOST) cr[m++] ← t;
if (m) {
  for (int t ← 0; t < cr[0]; t++) {
    cost += topgap[t];
    reach(toppk[t]);
    sreach(topsc[t]);
  }
  for (int t ← cr[m − 1]; t < nt − 1; t++) {
    cost += topgap[t];
    reach(toppk[t]);
    sreach(topsc[t]);
  }
  int k ← m/2;
  NCL ← 0;
  for (int i ← 0; i < k; i++) {
    int g[2 * MAXL], pk[2 * MAXL], scs[2 * MAXL], n ← 0, m1, p1, s1;
    int a ← cr[i], b ← (i < k − 1) ? cr[i + 1] : cr[m − 1 − i];
    for (int t ← a; t < b; t++) {
      g[n] ← topgap[t];
      scs[n] ← topsc[t];
      pk[n++] ← toppk[t];
    }
    if (i < k − 1)
      for (int t ← cr[m − 2 − i]; t < cr[m − 1 − i]; t++) {
        g[n] ← topgap[t];
        scs[n] ← topsc[t];
        pk[n++] ← toppk[t];
      }
    cost += cycle(g, pk, scs, n, &m1, &p1, &s1);
    if (i ≡ k − 1) {      /* only a path across the middle (A_k .. rho A_k) can join some X to rho X: a
        ring's right side is just the image of its left side */
      CLD[NCL] ← m1;
      CLS[NCL] ← s1;
      CLP[NCL++] ← p1;
    }
  }
  if (m & 1) NCL ← 0;      /* self class present */
}
else {
  for (int t ← 0; t < nt − 1; t++) {
    cost += topgap[t];
    reach(toppk[t]);
    sreach(topsc[t]);
  }
}
```

This code is used in chunk 61.

66 ¶ ⟨ Add the self term and the terms for the side and the top 66 ⟩ ≡

```
DEDGE ← 0;
if (¬tch) {
  int d1 ← rs[0], d2 ← L − 1 − re[R − 1];
  DEDGE ← d1 < d2 ? d1 : d2;
}      /* without a self class the middle cycle must be closed completely: pay its largest gap, which then
    also counts for the height */
if (NCL) {
```

```
    int ex ← 1 ≪ 20;
    for (int c ← 0; c < NCL; c++) {
      int e2 ← CLD[c] + height_extra(CLP[c] > H0 ? CLP[c] : H0, CLS[c] < S0 ? CLS[c] : S0);
      if (e2 < ex) ex ← e2;
    }
    cost += ex;
  }
  else cost += height_extra(H0, S0);
```

This code is used in chunk 61.

¶ An older and weaker bound is retained for comparison (option `-K`). It computes a minimum spanning tree over the gaps by Kruskal's algorithm, identifying partners, and adds the distance to the side. It ignores the kink, the self term, and the height.

67 ⟨ Subroutines 9 ⟩ +≡

```
static int exact_bound(const uint8_t *key, int tch)
{
  int sup[MAXL + 1];
  int occ[MAXL], no ← 0;
  for (int j ← 1; j ≤ MAXC; j++) {
    int q ← key[L + j − 1];
    if (¬q) break;
    sup[j] ← (q ≠ PSELF ∧ q ≠ PGHOST ∧ q < j) ? q : j;
  }
  for (int i ← 0; i < L; i++)
    if (key[i]) occ[no++] ← i;
  if (¬no) return 0;      /* Kruskal over gaps between consecutive occupied positions */
  int par[MAXL + 1];
  int ncls ← 0;
  int seen[MAXL + 1];
  memset(seen, 0, sizeof (seen));
  for (int a ← 0; a < no; a++) {
    int s ← sup[key[occ[a]]];
    if (¬seen[s]) {
      seen[s] ← 1;
      ncls++;
    }
    par[s] ← s;
  }
  int cost ← 0;
  if (ncls > 1) {      /* counting sort of gaps by weight */
    int ne ← no − 1;
    int wgt[MAXL], ord[MAXL], bucket[MAXL + 1];
    memset(bucket, 0, sizeof (bucket));
    for (int a ← 0; a < ne; a++) {
      wgt[a] ← occ[a + 1] − occ[a] − 1;
      bucket[wgt[a]]++;
    }
    for (int g ← 1; g ≤ L; g++) bucket[g] += bucket[g − 1];
    for (int a ← ne − 1; a ≥ 0; a−−) ord[−−bucket[wgt[a]]] ← a;
    for (int e ← 0; e < ne ∧ ncls > 1; e++) {
      int a ← ord[e];
      int x ← sup[key[occ[a]]], y ← sup[key[occ[a + 1]]];
      while (par[x] ≠ x) x ← par[x];
      while (par[y] ≠ y) y ← par[y];
      if (x ≠ y) {
        par[x] ← y;
```

```
        cost += wgt[a];
        ncls--;
      }
    }
  }
  if (¬tch) {
    int d1 ← occ[0], d2 ← L − 1 − occ[no − 1];
    cost += d1 < d2 ? d1 : d2;
  }
  return cost;
}
```

¶ The function *bound* returns a lower bound on the *weight* that must still be added to a nonempty state, or -1 if the state can never be completed. Here k is the number of the current row, counted from the first row of H, and *kmin* is the smallest number of rows for which the height reaches L. The cactus bound is not valid in the middle of the twin row, because there the adjacencies to the image row are not yet all known.

68 ⟨ Subroutines 9 ⟩ +≡

```
static int bound(const uint8_t *key, int nt, int k, int kmin, int i, int last, int first_twin)
{
  int lb ← kmin > k ? 2 * (kmin − k) : 0;
  if (¬nt) {
    int dist ← L;
    for (int j ← 0; j < L; j++)
      if (key[j]) {
        int dd ← j < L − 1 − j ? j : L − 1 − j;
        if (dd < dist) dist ← dd;
      }
    if (2 * dist > lb) lb ← 2 * dist;
  }
  if (¬first_twin ∨ last) {
    if (¬completion_possible(key)) return −1;
    int kink ← (¬last ∧ ¬key[i] ∧ i + 1 < L ∧ key[i + 1]) ? i + 1 : −1;
    CUR_U ← U0 + k − 1;
    USTAR ← U0 + kmin − 1;
    int eb ← 2 * (OLDBOUND ? exact_bound(key, nt) : cactus_bound(key, nt, kink));
    if (eb > lb) lb ← eb;
  }
  return lb;
}
```

7 The main loop

The results of a case are collected in two vectors, indexed by n: `LT` for vspan < uspan and `EQ` for vspan = uspan.

69 ⟨ Global variables 8 ⟩ +≡

```
static cnt_t *GF;      /* dense scratch vector of counters */
static cnt_t *LT, *EQ;      /* results of one case */
static size_t MAXSTATES, MAXBYTES, MAXRECS, MAXKW;      /* statistics */
```

¶ When a polyomino is complete after *ktop* rows, it is counted if it touches the sides and is at least as high as wide.

70 ⟨ Subroutines 9 ⟩ +≡

```
static void record(int ktop, const cnt_t *gf, int tch)
{
  int h ← uspan(ktop);
```

```
  if (¬tch ∨ h < L) return;
  cnt_t *tgt ← h ≡ L ? EQ : LT;
  for (int wt ← 0; wt ≤ NMAX; wt++)
    if (gf[wt]) tgt[wt] ← addm(tgt[wt], gf[wt]);
}
```

¶ Progress is reported on the standard error stream, after every row (`-v 1`, the default) or after every cell (`-v 2`). With `-v 1`, a line is also written within a row when more than `PSEC` seconds have passed since the last line. Every line is flushed immediately. The field "done" is the number of polyominoes that the current case has completed so far (modulo M).

71 ⟨ Global variables 8 ⟩ +≡

```
static int VERB ← 1;      /* -v: 0 none, 1 per row, 2 per cell */
static double PSEC ← 60;      /* -p: within a row, a line at most every PSEC s */
static int G_C, G_D, G_VHI;
static double T_START, T_LAST;
```

72 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static double wall(void)
{
  struct timespec ts;
  clock_gettime(CLOCK_MONOTONIC, &ts);
  return ts.tv_sec + 1 · 10^-9 * ts.tv_nsec;
}
static void stat_line(const char *tag, int k, int u, int ci, int nu, const table_t *cur, const table_t
        *nxt)
{
  double now ← wall( );
  T_LAST ← now;
  cnt_t done ← 0;
  for (int n ← 0; n ≤ NMAX; n++) done ← addm(done, addm(LT[n], EQ[n]));
  fprintf(stderr, "%s c=%d d=%d vhi=%d L=%d row k=%d u=%d cell %d/%d: ""%zu states, %zu rec\
      ords, %.1f MB, done %s, %.1fs wall, %.1fs cpu\n", tag, G_C, G_D, G_VHI, L, k, u, ci, nu,
      cur→n, cur→nrec, (tab_bytes(cur) + tab_bytes(nxt))/1 · 10^6, cs(done), now − T_START,
      (double) clock( )/CLOCKS_PER_SEC);
  fflush(stderr);
}
```

¶ Here is the function that treats one case.

73 ⟨ Subroutines 9 ⟩ +≡

```
static void count_case(int c, int d, int vhi)
{
  ⟨ Set up the geometry of the case 74 ⟩
  ⟨ Local variables of count_case 75 ⟩
  ⟨ Create the initial states 76 ⟩
  int cells_u[MAXL];
  for (int k ← 1; cur→n; k++) {
    ⟨ Determine the cells of row k 77 ⟩
    for (int ci ← 0; ci < nu; ci++) {
      ⟨ Decide cell ci of the row for all states 78 ⟩
    }
    ⟨ Merge every state with its mirror image 82 ⟩
    if (VERB ≥ 1) stat_line("#row ", k, u, nu, nu, cur, nxt);
  }
  tab_free(&A);
  tab_free(&B);
}
```

¶ The strip with a single column is a special case.

74 ⟨ Set up the geometry of the case 74 ⟩ ≡
```
G_C ← c, G_D ← d, G_VHI ← vhi;
S ← c + d;
T ← c − d;
vlo ← T − vhi;
L ← vhi − vlo + 1;
E ← S & 1;
memset(LT, 0, NW ∗ sizeof(cnt_t));
memset(EQ, 0, NW ∗ sizeof(cnt_t));
if (L ≡ 1) {
  if (c ≡ 0 ∧ d ≡ 0) EQ[1] ← addm(EQ[1], 1);
  return;
}
if (L > MAXL) {
  fprintf(stderr, "strip too wide\n");
  exit(1);
}
MAXC ← (L + 1)/2 + 1;
GOFF ← MAXC + 1;
KL ← L + MAXC + 1;
KW ← (3 ∗ L + MAXC + 1 + 63)/64;
NW ← NMAX + 1;
int kmin ← 0;
while (uspan(kmin) < L) kmin++;
```
This code is used in chunk 73.

¶ Two tables are used alternately: *cur* for the states before the current cell and *nxt* for the states after it.

75 ⟨ Local variables of *count_case* 75 ⟩ ≡
```
table_t A, B, ∗cur ← &A, ∗nxt ← &B;
tab_init(&A, 1024);
tab_init(&B, 1024);
uint8_t key[MAXL ∗ 2 + 4];
cnt_t one[MAXL ∗ 2 + 4];
work_t w;
int u0;
```
This code is used in chunk 73.

¶ **Initial states.** If s is even, the middle row $u_m = s/2$ is its own image. We enumerate all nonempty subsets of it that are symmetric under $i \leftrightarrow L-1-i$. Every cell is a class of its own; the partner of a cell is its mirror cell, and the centre cell, if present, is a self class. The weight is the number of cells. If s is odd, we start with the empty state before the twin row.

76 ⟨ Create the initial states 76 ⟩ ≡
```
if (E ≡ 0) {
  int um ← S/2;
  int half[MAXL], nh ← 0;
  for (int i ← 0; i < L; i++)
    if (((vlo + i − um) % 2 + 2) % 2 ≡ 0 ∧ 2 ∗ (vlo + i) ≥ T) half[nh++] ← i;
        /∗ all nonempty subsets of half with at most NMAX cells in the row ∗/
  for (uint64_t mask ← 1; mask < (1ULL ≪ nh); mask++) {
    int cells[MAXL], nc ← 0, isc[MAXL];
    memset(isc, 0, sizeof(int) ∗ L);
    for (int b ← 0; b < nh; b++)
      if (mask ≫ b & 1) {
        isc[half[b]] ← 1;
        isc[L − 1 − half[b]] ← 1;
```

```
    }
  for (int i ← 0; i < L; i++)
    if (isc[i]) cells[nc++] ← i;
  if (nc > NMAX) continue;
  int lab[MAXL];
  for (int a ← 0; a < MAXN; a++) w.parent[a] ← a;
  for (int i ← 0; i < L; i++) w.f[i] ← 0;
  for (int a ← 0; a < nc; a++) {
    lab[cells[a]] ← a + 1;
    w.f[cells[a]] ← a + 1;
  }
  for (int a ← 0; a < nc; a++) {
    int i ← cells[a], j ← L − 1 − i;
    w.partner[a + 1] ← j ≡ i ? −1 : lab[j];
  }
  int tch ← isc[0] ∨ isc[L − 1];
  encode(&w, key, tch);
  memset(one, 0, sizeof(cnt_t) ∗ NW);
  one[nc] ← 1;
  insert_pruned(cur, key, one, 0, 0);
 }
 u0 ← um + 1;
 U0 ← u0;
}
else {
 memset(key, 0, KL);
 memset(one, 0, sizeof(cnt_t) ∗ NW);
 one[0] ← 1;
 insert_pruned(cur, key, one, 0, 0);
 u0 ← (S + 1)/2;
 U0 ← u0;
}
```

This code is used in chunk 73.

¶ The cells of row u are at the positions i for which $v_{\min} + i - u$ is even.

77 ⟨ Determine the cells of row k 77 ⟩ ≡

```
int u ← u0 + k − 1, nu ← 0;
for (int i ← 0; i < L; i++)
  if (((vlo + i − u) % 2 + 2) % 2 ≡ 0) cells_u[nu++] ← i;
int first_twin ← (E ≡ 1 ∧ k ≡ 1);
```

This code is used in chunk 73.

78 ¶ ⟨ Decide cell ci of the row for all states 78 ⟩ ≡

```
int i ← cells_u[ci];
int last ← ci ≡ nu − 1;
tab_clear(nxt);
for (size_t si ← 0; si < cur→n; si++) {
  ⟨ Extend state si by the two choices for cell i 79 ⟩
}
⟨ Swap the tables and update the statistics 81 ⟩
```

This code is used in chunk 73.

¶ The empty state exists only in the twin row. A polyomino that is completed while row k is processed has its top in row $k - 1$.

79 ⟨ Extend state *si* by the two choices for cell *i* 79 ⟩ ≡

```
uint8_t sk[MAXL * 2 + 4];
unpack(cur→keys + si * KW, sk);
const cnt_t *gf ← GF;
tab_load(cur, si, GF);
int tch ← sk[KL − 1];
int empty ← 1;
for (int j ← 0; j < L; j++)
  if (sk[j]) {
    empty ← 0;
    break;
  }
for (int occ ← 0; occ ≤ 1; occ++) {
  if (occ ∧ k > 1 ∧ empty) continue;
  int nt, m;
  int status ← step(&w, sk, tch, i, occ, last, first_twin, key, &nt, &m);
  if (status ≡ DEAD) continue;
  if (status ≡ COMPLETE) {
    record(k − 1, gf, tch);
    continue;
  }
  int add ← occ ? 2 : 0;
  if (m ≡ 0) {
    if (first_twin ∧ ¬last) insert_pruned(nxt, key, gf, add, 0);
    continue;
  }
  int lb ← bound(key, nt, k, kmin, i, last, first_twin);
  if (lb < 0) continue;
  ⟨ Sharpen lb by looking one cell ahead 80 ⟩
  insert_pruned(nxt, key, gf, add, lb);
}
tab_unload(cur, si, GF);
```

This code is used in chunk 78.

¶ **Lookahead.** The bound does not know which cells of the current row are already decided to be empty (Figure 4). This matters most at the kink: if the cell i is empty and there is a frontier cell right of it, that cell depends entirely on the next cell $i2$ of the row. We therefore try both possibilities for the next cell and take the smaller of the two bounds, including the weight of the cell. A completion in the lookahead counts only if it would be a valid polyomino. With the option `-A` the lookahead is done for every cell, and with `-N` it is switched off.

80 ⟨ Sharpen *lb* by looking one cell ahead 80 ⟩ ≡

```
if (LOOKAHEAD ∧ ¬last ∧ ¬first_twin ∧ (LOOKAHEAD ≡ 2 ∨ (¬key[i] ∧ i + 1 < L ∧ key[i + 1]))) {
    /* one step ahead: the next cell is present or not */
  int i2 ← cells_u[ci + 1], last2 ← ci + 1 ≡ nu − 1;
  int la ← 1 ≪ 20;
  uint8_t k2[MAXL * 2 + 4];
  for (int o2 ← 0; o2 ≤ 1 ∧ la; o2++) {
    int nt2, m2;
    int st2 ← step(&w, key, nt, i2, o2, last2, 0, k2, &nt2, &m2);
    if (st2 ≡ DEAD) continue;      /* a completion only counts if it is a valid polyomino */
    if (st2 ≡ COMPLETE ∧ ¬(nt ∧ uspan(k − 1) ≥ L)) continue;
    int val ← 2 * o2;
    if (st2 ≡ OK) {
      if (m2 ≡ 0) continue;
      int lb2 ← bound(k2, nt2, k, kmin, i2, last2, 0);
```

```
        if (lb2 < 0) continue;
        val += lb2;
      }
      if (val < la) la ← val;
    }
    if (la > lb) lb ← la;
  }
```

This code is used in chunk 79.

81 ¶ ⟨ Swap the tables and update the statistics 81 ⟩ ≡

```
table_t *x ← cur;
cur ← nxt;
nxt ← x;
if (cur→n > MAXSTATES) MAXSTATES ← cur→n;
size_t by ← tab_bytes(cur) + tab_bytes(nxt);
if (by > MAXBYTES) MAXBYTES ← by, MAXRECS ← cur→nrec, MAXKW ← KW;
if (¬last ∧ (VERB ≥ 2 ∨ (VERB ≡ 1 ∧ wall( ) − T_LAST ≥ PSEC)))
  stat_line("#cell", k, u, ci + 1, nu, cur, nxt);
```

This code is used in chunk 78.

¶ **Reflection.** If $t = 0$, the strip is symmetric under $v \to -v$, and at the end of a row the mirror image of a state is again a valid state with the same number of completions. We merge each state with its mirror image by keeping the lexicographically smaller byte key. This almost halves the number of states for these cases. (The option `-R` disables it.) Positions are reversed in the working state, and *encode* then renumbers the classes.

82 ⟨ Merge every state with its mirror image 82 ⟩ ≡

```
if (REFLECT ∧ T ≡ 0 ∧ cur→n) {      /* merge each state with its mirror image v -> -v */
  tab_clear(nxt);
  uint8_t rkey[MAXL * 2 + 4];
  for (size_t si ← 0; si < cur→n; si++) {
    uint8_t sk[MAXL * 2 + 4];
    unpack(cur→keys + si * KW, sk);
    decode(&w, sk);
    for (int a ← 0, b ← L − 1; a < b; a++, b−−) {
      int x ← w.f[a];
      w.f[a] ← w.f[b];
      w.f[b] ← x;
    }
    encode(&w, rkey, sk[KL − 1]);
    const uint8_t *use ← memcmp(rkey, sk, KL) < 0 ? rkey : sk;
    tab_load(cur, si, GF);
    insert_pruned(nxt, use, GF, 0, 0);
    tab_unload(cur, si, GF);
  }
  table_t *x ← cur;
  cur ← nxt;
  nxt ← x;
}
```

This code is used in chunk 73.

8 The main program

```
-m M   count modulo M (default: modulo 2^64, or 2^128 with CNT128)
-o F   append the results to the file F (default results180.py)
```

```
-v V   progress report: 0 none, 1 per row (default), 2 per cell
-p S   with -v 1, report within a row every S seconds (default 60)
-R     do not merge mirror states
-N     no lookahead;   -A  lookahead after every cell
-H     no height terms in the bound
-K     use the old bound (minimum spanning tree only)
-C     check that packing and unpacking of every key are consistent
```

83 ⟨ Global variables 8 ⟩ +≡

```
static int REFLECT ← 1;      /* merge mirror states? */
static int OLDBOUND ← 0;     /* -K */
static const char *RESFILE ← "results180.py";     /* -o */
static int LOOKAHEAD ← 1;     /* 0 none, 1 at the kink, 2 after every cell */
static int CHECK ← 0;     /* -C */
```

84 ¶ ⟨ The main program 84 ⟩ ≡

```
int main(int argc, char **argv)
{
  ⟨ Parse the command line 85 ⟩
  clock_t t0 ← clock( );
  T_START ← T_LAST ← wall( );
  ⟨ Print the program parameters 86 ⟩
  if (argc − a ≡ 4) {
    ⟨ Count a single case and print the result 87 ⟩
  }
  else {
    ⟨ Count all cases and add them up by type of centre 88 ⟩
    ⟨ Append the results to the results file 89 ⟩
    ⟨ Print the table of results 90 ⟩
  }
  ⟨ Print the final statistics 91 ⟩
  return 0;
}
```

This code is used in chunk 2.

85 ¶ ⟨ Parse the command line 85 ⟩ ≡

```
int a ← 1;
while (a < argc ∧ argv[a][0] ≡ '-') {
  if (¬strcmp(argv[a], "-m") ∧ a + 1 < argc) MOD ← strtoull(argv[++a], 0, 10), a++;
  else if (¬strcmp(argv[a], "-o") ∧ a + 1 < argc) RESFILE ← argv[++a], a++;
  else if (¬strcmp(argv[a], "-A")) LOOKAHEAD ← 2, a++;
  else if (¬strcmp(argv[a], "-N")) LOOKAHEAD ← 0, a++;
  else if (¬strcmp(argv[a], "-H")) HEIGHT ← 0, a++;
  else if (¬strcmp(argv[a], "-K")) OLDBOUND ← 1, a++;
  else if (¬strcmp(argv[a], "-C")) CHECK ← 1, a++;
  else if (¬strcmp(argv[a], "-v") ∧ a + 1 < argc) VERB ← atoi(argv[++a]), a++;
  else if (¬strcmp(argv[a], "-p") ∧ a + 1 < argc) PSEC ← atof(argv[++a]), a++;
  else if (¬strcmp(argv[a], "-R")) REFLECT ← 0, a++;
  else break;
}
if (a < argc ∧ argv[a][0] ≡ '-') {
  fprintf(stderr, "%s: unknown option %s\n", argv[0], argv[a]);
  return 1;
}
if (a ≥ argc) {
  fprintf(stderr, "usage: %s [-m M] [-R] N [c d vhi]\n", argv[0]);
  return 1;
```

```
}
NMAX ← atoi(argv[a]);
NW ← NMAX + 1;
LT ← calloc(NW, sizeof(cnt_t));
GF ← calloc(NW, sizeof(cnt_t));
EQ ← calloc(NW, sizeof(cnt_t));
```

This code is used in chunk 84.

¶ Both output streams start with the parameters of the run, beginning with the complete command line (including the name *argv*[0] of the program), so that every log file and every result file is self-describing.

86 ⟨ Print the program parameters 86 ⟩ ≡

```
for (int str ← 0; str < 2; str++) {
  FILE *f ← str ? stdout : stderr;
  fprintf(f, "# command:");
  for (int q ← 0; q < argc; q++) fprintf(f, " %s", argv[q]);
  fprintf(f, "\n# diag180 built %s %s; N=%d", __DATE__, __TIME__, NMAX);
  if (argc − a ≡ 4) fprintf(f, ", single case c=%s d=%s vhi=%s", argv[a + 1], argv[a + 2], argv[a + 3]);
  if (MOD) fprintf(f, ", modulus %llu\n", (unsigned long long) MOD);
  else fprintf(f, ", modulus 2**"CNT_BITS"\n");
  fprintf(f,
      "# bound: %s, height term %s, lookahead %s; reflection merge %s; pack check %s\n",
      OLDBOUND ? "old Kruskal (-K)" : "cactus", HEIGHT ? "on" : "off (-H)",
      LOOKAHEAD ≡ 2 ? "every cell (-A)" : LOOKAHEAD ? "at the kink" : "off (-N)",
      REFLECT ? "on" : "off (-R)", CHECK ? "on (-C)" : "off");
  fprintf(f, "# output: results file %s, progress -v %d -p %g s\n", RESFILE, VERB, PSEC);
  fprintf(f, "# sizes: counter %zu bytes, count record %zu bytes, key word 64 bits\n",
      sizeof(cnt_t), sizeof(rec_t));
  fflush(f);
}
```

This code is used in chunk 84.

¶ A single case can be run separately, for example for distributing the cases over the nodes of a cluster.

87 ⟨ Count a single case and print the result 87 ⟩ ≡

```
int c ← atoi(argv[a + 1]), d ← atoi(argv[a + 2]), vhi ← atoi(argv[a + 3]);
count_case(c, d, vhi);
if (MOD)
  printf("# modulus %llu; n #(vspan<uspan) #(vspan=uspan)\n", (unsigned long long) MOD);
else printf("# modulus 2**"CNT_BITS"; n #(vspan<uspan) #(vspan=uspan)\n");
for (int n ← 1; n ≤ NMAX; n++) printf("%d %s %s\n", n, cs(LT[n]), cs(EQ[n]));
```

This code is used in chunk 84.

¶ The cases are $(c, d) \in \{(0, 0), (1, 0), (1, 1)\}$ and all $v_{\max}$ for which $L \le N$. The edge-centred case $(1, 0)$ is counted twice, since it stands also for its mirror image $(0, 1)$.

88 ⟨ Count all cases and add them up by type of centre 88 ⟩ ≡

```
    /* centre types: 0 cell (c=d=0), 1 edge midpoint (c != d), 2 vertex */
cnt_t *out ← calloc(3 * NW, sizeof(cnt_t));
for (int c ← 0; c ≤ 1; c++)
  for (int d ← 0; d ≤ 1; d++) {      /* (c, d) = (0, 1) is the mirror image (x <-> y, v -> -v) of (1, 0)
        with vhi one smaller: same counts, so count (1, 0) twice instead */
    if (c ≡ 0 ∧ d ≡ 1) continue;
    int t ← c − d, ty ← c + d, twice ← c ≠ d;
    int vhi ← t ≥ 0 ? (t + 1)/2 : 0;
```

```
    for ( ; 2 * vhi - t + 1 ≤ NMAX; vhi++) {
      if (2 * vhi - t + 1 < 1) continue;
      count_case(c, d, vhi);
      for (int n ← 1; n ≤ NMAX; n++) {
        cnt_t *o ← &out[ty * NW + n];
        *o ← addm(*o, addm(LT[n], LT[n]));
        *o ← addm(*o, EQ[n]);
        if (twice) {
          *o ← addm(*o, addm(LT[n], LT[n]));
          *o ← addm(*o, EQ[n]);
        }
      }
    }
  }
```

This code is used in chunk 84.

¶ The results are appended to a file in Python syntax, with lines of the form

```
Results[n, m, 'R180C'] = x
```

where m is the modulus. Runs with different moduli accumulate in the same file, and the final combination (Chinese remaindering, Burnside's lemma) can be done in Python with integers of unlimited size.

89 ⟨ Append the results to the results file 89 ⟩ ≡

```
/* results file for the final combination (CRT, Burnside) in Python; appended, so runs with different moduli accumulate */
FILE *rf ← fopen(RESFILE, "a");
if (¬rf) {
  perror(RESFILE);
  return 1;
}
char mstr[32];
if (MOD) snprintf(mstr, sizeof mstr, "%llu", (unsigned long long) MOD);
else snprintf(mstr, sizeof mstr, "2**"CNT_BITS);
fprintf(rf,
    "# diag180 N=%d modulus %s\ntry:\n    Results\nexcept NameError:\n    Results = {}\n",
    NMAX, mstr);
static const char *tyname[3] ← {"R180C", "R180M", "R180V"};
for (int n ← 1; n ≤ NMAX; n++)
  for (int ty ← 0; ty < 3; ty++)
    fprintf(rf, "Results[%d, %s, '%s'] = %s\n", n, mstr, tyname[ty], cs(out[ty * NW + n]));
fclose(rf);
printf("# modulus %s; results appended to %s\n", mstr, RESFILE);
```

This code is used in chunk 84.

¶ The table of results has a line for each n with the total $\mathrm{fix}_\rho(n)$ and its three parts for the three types of centres.

90 ⟨ Print the table of results 90 ⟩ ≡

```
printf("# n fix(r180) R180C(centre in a cell) R180M(edge midpoint) R180V(vertex)\n");
for (int n ← 1; n ≤ NMAX; n++) {
  cnt_t tot ← addm(addm(out[n], out[NW + n]), out[2 * NW + n]);
  printf("%d %s %s %s\n", n, cs(tot), cs(out[n]), cs(out[NW + n]), cs(out[2 * NW + n]));
}
```

This code is used in chunk 84.

91 ¶ ⟨ Print the final statistics 91 ⟩ ≡
fprintf(*stderr*,
"# peak memory %.1f MB (both tables; %zu count records, key %zu 64-bit words)\n",
MAXBYTES/1 · 10^6, MAXRECS, MAXKW);
fprintf(*stderr*, "# max states %zu, %.2fs\n", MAXSTATES, (**double**)(*clock*( ) − *t0*)/CLOCKS_PER_SEC);
This code is used in chunk 84.

9 Results and experiments

Correctness. The program has been checked in several ways. The results agree with a direct enumeration of all polyominoes by Redelmeier's algorithm [10] for $n \le 16$, and with an independent Python prototype of the same method for $n \le 36$. Runs with different moduli are consistent. Different variants of the bound (`-K`, `-H`, `-N`, `-A`) lead to the same counts; this is a strong test, since an invalid bound would lose polyominoes.

Tightness of the bound. For all states that arise for $N \le 22$, after every cell, we have computed the true minimum weight of a completion by exhaustive search. The bound, together with the lookahead at the kink, was equal to the true value for every one of the 87,842 states. Thus, at least in this range, no state is stored in vain.

Growth. Table 1 shows the maximum number of states in a table and the running time. The number of states grows by a factor of about 1.25 when N increases by one. The run for $N = 60$ took 66 minutes on a single core and needed 304 MB (before the mirror-image case $(0, 1)$ was eliminated, which saves one third of the time).

N	max. states	records	MB	time (s)
16	70	146	0.0	0.01
20	219	409	0.0	0.03
24	550	1,225	0.1	0.16
28	1,417	3,350	0.2	0.62
32	3,674	8,308	0.4	1.72
36	9,793	20,223	1.1	5.34
40	21,191	44,777	2.8	16.32
44	55,629	130,869	7.4	49.63
48	147,608	352,534	21.3	149.08
60	2,137,770	4,689,188	304.1	3991 (other machine, see text)

Table 1: Maximum number of states in one table, number of count records at the peak, memory for both tables, and running time on a single core (64-bit counters).

Values. Table 2 lists $\mathrm{fix}_\rho(n)$ for $n \le 60$.

10 References

References

[1] Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. *Algorithmica* **88** (2026), Article 51. https://doi.org/10.1007/s00453-026-01392-6. Preliminary version in: *Proc. Symposium on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2024, pp. 133–143.

[2] Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited: Corrigendum and addendum. Preprint, arXiv:2610.10632, October 2026.

[3] Gill Barequet, Micha Moffie, Ares Ribó, and Günter Rote. Counting polyominoes on twisted cylinders. *Integers* **6** (2006), Article A22.

[4] Iwan Jensen. Enumerations of lattice animals and trees. *Journal of Statistical Physics* **102** (2001), 865–881.

[5] Iwan Jensen. Counting polyominoes: a parallel implementation for cluster computing. In: *Computational Science — ICCS 2003*, Lecture Notes in Computer Science 2659, Springer, 2003, pp. 203–212.

n	$\mathrm{fix}_\rho(n)$	n	$\mathrm{fix}_\rho(n)$
1	1	31	223,859,532
2	2	32	880,936,564
3	2	33	879,285,686
4	7	34	3,471,406,866
5	7	35	3,460,424,846
6	24	36	13,701,930,007
7	24	37	13,642,112,667
8	85	38	54,162,590,412
9	86	39	53,865,266,960
10	310	40	214,385,586,434
11	316	41	212,982,833,863
12	1,157	42	849,603,176,618
13	1,178	43	843,202,561,450
14	4,374	44	3,370,654,595,766
15	4,442	45	3,342,114,787,290
16	16,677	46	13,385,948,957,670
17	16,896	47	13,260,804,930,438
18	64,030	48	53,209,001,488,282
19	64,716	49	52,667,141,592,300
20	247,196	50	211,685,231,147,940
21	249,283	51	209,361,911,445,176
22	958,630	52	842,825,330,054,404
23	964,708	53	832,940,890,785,598
24	3,731,592	54	3,358,159,320,114,116
25	3,748,031	55	3,316,370,384,325,374
26	14,572,036	56	13,389,349,169,914,498
27	14,610,484	57	13,213,587,492,371,313
28	57,060,549	58	53,418,553,023,401,386
29	57,118,440	59	52,682,486,874,469,758
30	223,968,274	60	213,246,176,582,622,258

Table 2: The number of fixed polyominoes with n cells that are invariant under a rotation by 180°.

[6] Donald E. Knuth. POLYNUM. CWEB program, 2016. https://cs.stanford.edu/~knuth/programs.html

[7] Donald E. Knuth. POLYSLAVE. CWEB program, 2024. https://cs.stanford.edu/~knuth/programs.html

[8] Donald E. Knuth and Silvio Levy. *The CWEB System of Structured Documentation*. Addison-Wesley, 1994.

[9] OEIS Foundation. *The On-Line Encyclopedia of Integer Sequences*, sequences A000105 and A001168. https://oeis.org

[10] D. Hugh Redelmeier. Counting polyominoes: yet another attack. *Discrete Mathematics* **36** (1981), 191–203.

[11] Toshihiro Shirakawa. Enumeration of polyominoes up to size $N = 59$. Preprint, arXiv:2510.22446, October 2025.

Contents

1 Introduction **1**
1.1 Symmetry classes and cases 2
1.2 Diagonal coordinates 2
1.3 The half polyomino 3

2 The state **4**
2.1 Byte keys . . . 4
2.2 The working state . . . 5
2.3 Removing a cell from the frontier . . . 7
2.4 A transition . . . 8

3 Packed keys **8**

4 Counters **10**

5 The tables **11**

6 The lower bound **15**
6.1 Connections on the diagonal lattice . . . 15
6.2 The cactus of gaps . . . 15
6.3 Reaching the sides and the top . . . 16
6.4 Putting the bound together . . . 19

7 The main loop **23**

8 The main program **28**

9 Results and experiments **32**

10 References **32**

Index

__DATE__: 86.
__int128: 32.
__TIME__: 86.
A: 75.
a: 16 17 18 19 20 31 34 57 64 65 67 76 82 85.
add: 44 79.
addm: 34 42 70 72 74 88 90.
any: 23 44.
argc: 84 85 86.
argv: 84 85 86 87.
atof: 85.
atoi: 85 87.
B: 75.
b: 18 19 34 64 65 76.
best: 57.
both_cost: 56 57 63.
bound: 68 79 80.
bucket: 67.
buf: 35.
by: 81.
c: 36 42 57 66 73 87 88.
C_CONTINUE: 26 27 30 31.
C_EMPTY: 26 27 30 31.
C_FIRST: 26 27 30 31.
C_LAST: 26 27 30 31.
C_MIDDLE: 26 27 30 31.
C_SINGLETON: 26 27 30 31.
cactus_bound: 61 68.
calloc: 38 40 85 88.
cap: 36 38 40 41.
cells: 76.
cells_u: 73 77 78 80.
CHECK: 41 83 85 86.
ci: 72 73 78 80 81.
CLD: 53 65 66.
clock: 72 84 91.
clock_gettime: 72.
CLOCK_MONOTONIC: 72.
CLOCKS_PER_SEC: 72 91.
CLP: 53 65 66.
CLS: 53 65 66.
CNT_BITS: 32 86 87 89.
cnt_t: 32 34 35 36 42 43 44 69 70 72 74 75 76 79 85 86 88 90.
CNT128: 32.
code: 30 31.
COMPLETE: 22 23 24 79 80.
completion_possible: 46 68.
cost: 61 64 65 66 67.
count_case: 73 87 88.
cr: 65.
cs: 35 72 87 89 90.
cur: 72 73 75 76 78 79 81 82.
CUR_TCH: 53 59 63.
CUR_U: 53 54 68.
cycle: 50 58 64 65.
d: 73 87 88.
dd: 68.
DEAD: 22 23 24 79 80.
decode: 20 24 82.
DEDGE: 53 59 66.
dist: 68.
done: 72.
d1: 66 67.
d2: 66 67.
E: 8.
e: 35 56 57 67.
eb: 68.
empty: 79.
encode: 21 24 76 82.
EQ: 69 70 72 74 85 87 88.
ex: 60 66.
exact_bound: 67 68.
exit: 37 40 41 74.
e2: 60 66.
f: 14 86.
fclose: 89.
fflush: 72 86.
find: 16 17 18 21 23.
finish: 59 60.
first: 30.
first_twin: 24 25 68 77 79 80.
fopen: 89.
fprintf: 37 40 41 72 74 85 86 89 91.
free: 38 40.
frow: 54 63 64.
g: 58 64 65 67.
G_C: 71 72 74.
G_D: 71 72 74.
G_VHI: 71 72 74.
gap_score: 57 64.
gb: 64.
getbits: 29 31.
gf: 43 44 70 79.
GF: 69 79 82 85.
GH: 14 17.
GOFF: 14 15 17 74.
h: 38 39 40 55 56 59 60 70.
half: 76.
hash: 36 38 40 41.
head: 36 38 40 41 42 43.
HEIGHT: 53 59 85 86.
height_extra: 60 66.
hkey: 39 40 41.
hmask: 36 38 40 41.
HO: 53 55 63 66.
i: 20 21 23 24 29 30 31 39 40 41 62 63 65 67 68 76 77 78.
insert_pruned: 44 76 79 82.
isc: 76.
ISGHOST: 14 17 18 23.
i2: 80.
j: 20 21 23 30 31 46 54 56 57 64 67 68 76 79.
jj: 24.

k: 9 39 65 68 72 73.
key: 13 20 21 24 30 31 41 44 46 61 62 63 65 67 68 75 76 79 80.
keys: 36 38 40 41 79 82.
kink: 61 64 68.
KL: 13 21 30 31 41 74 76 79 82.
kmin: 68 74 79 80.
ktop: 70.
KW: 26 30 38 39 40 41 74 79 81 82.
k2: 41 80.
L: 8.
l: 30.
la: 80.
lab: 23 76.
last: 24 68 78 79 80 81.
lastrun: 30 62 64.
last2: 80.
lb: 44 68 79 80.
lb2: 80.
lim: 44.
*link_*: 18 19.
LOOKAHEAD: 80 83 85 86.
LT: 69 70 72 74 85 87 88.
m: 20 21 30 31 46 65 79.
main: 84.
map: 21.
mask: 76.
MAXBYTES: 69 81 91.
MAXC: 13 14 15 20 46 67 74.
MAXKW: 69 81 91.
MAXL: 8 14 21 30 31 41 53 62 64 65 67 73 74 75 76 79 80 82.
MAXN: 14 20 21 76.
MAXRECS: 69 81 91.
MAXSTATES: 69 81 91.
memcmp: 41 82.
memcpy: 41.
memset: 21 30 31 40 67 74 76.
MOD: 33 34 85 86 87 89.
mp: 24.
mstr: 89.
m1: 58 65.
m1p: 58.
m2: 58 80.
n: 36 58 65 72 87 88 89 90.
nb: 24 29 64.
nc: 76.
NCL: 53 63 65 66.
ncls: 67.
ne: 67.
next: 14 20 24 36 42 43.
nh: 76.
NMAX: 8 44 70 72 74 76 85 86 87 88 89 90.
no: 67.
NOPT: 53 58 60 63.
now: 72.
nrec: 36 38 40 42 72 81.
nt: 64 65 68 79 80.
ntp: 24.
nt2: 80.
nu: 72 73 77 78 80 81.
nw: 24.
NW: 8 74 76 85 88 89 90.
nxt: 72 73 75 78 79 81 82.
n1: 58.
o: 29 60 88.
occ: 24 67 79.
OK: 22 23 24 80.
OLDBOUND: 68 83 85 86.
one: 75 76.
OPTD: 51 53 58 60.
OPTP: 51 53 58 60.
OPTS: 51 53 58 60.
ord: 67.
out: 88 89 90.
o2: 80.
p: 17 23 35 37 40 41.
pack: 30 41.
par: 67.
parent: 14 16 18 20 76.
partner: 14 17 19 20 23 24 76.
pb: 64.
perror: 89.
PGHOST: 13 20 21 30 31 46 65 67.
pk: 58 64 65.
pos: 29 30 31.
printf: 87 89 90.
PSEC: 71 81 85 86.
PSELF: 13 20 21 31 67.
putbits: 29 30.
p1: 58 65.
p1p: 58.
q: 20 21 46 67 86.
qb: 64.
R: 62.
r: 21 23 35 42 43 56.
ra: 19 57.
rb: 19.
rcap: 36 38 42.
re: 62 64 66.
*re_*: 57.
reach: 50 55 58 63 65.
realloc: 37.
rec: 36 38 42 43.
rec_t: 36 38 42 86.
record: 70 79.
REFLECT: 82 83 85 86.
rep: 21.
RESFILE: 83 85 86 89.
retire: 23 24.
rf: 89.
rho: 17 19 21 23 24.
rkey: 82.
rl: 62 64.
rs: 62 64 66.
S: 8.
s: 34 37 59 60 67.
sb: 64.
sc: 55 58 64.
scs: 65.

seen: 67.
sep: 59.
si: 42 43 44 78 79 82.
sk: 24 79 82.
sl: 64.
snprintf: 89.
sp: 31 64.
sreach: 50 55 58 63 65.
stack: 31.
stat_line: 72 73 81.
status: 24 79.
stderr: 37 40 41 72 74 85 86 91.
stdout: 86.
step: 24 79 80.
str: 86.
strcmp: 85.
strtoull: 85.
st2: 80.
sum: 58.
sup: 67.
SO: 53 55 63 66.
s1: 58 65.
s1p: 58.
T: 8.
t: 38 40 41 42 43 44 58 65 88.
T_LAST: 71 72 81 84.
T_START: 71 72 84.
tab_add: 42 44.
tab_bytes: 38 72 81.
tab_clear: 40 78 82.
tab_free: 38 73.
tab_get: 41 44.
tab_grow: 40 41.
tab_init: 38 75.
tab_load: 43 79 82.
tab_unload: 43 79 82.
table_t: 36 38 40 41 42 43 44 72 75 81 82.
tag: 72.
tch: 21 24 61 63 64 66 67 70 76 79.
tgt: 70.
timespec: 72.
top: 59.
topgap: 64 65.
toplab: 64 65.
toppk: 64 65.
topsc: 64 65.
tot: 90.
ts: 72.
tv_nsec: 72.
tv_sec: 72.
twice: 88.
ty: 88 89.
tyname: 89.
t0: 84 91.
t1: 57.
t2: 57.
u: 72 77.
uint32_t: 36 38 40 42 43.
uint64_t: 29 30 31 32 33 36 39 41 76.
uint8_t: 20 21 24 30 31 36 41 44 46 61 67 68 75 79 80 82.
um: 76.
unite: 19 24.
unpack: 31 41 79 82.
use: 82.
uspan: 9 70 74 80.
USTAR: 53 56 59 68.
u0: 75 76 77.
UO: 53 68 76.
v: 29.
val: 80.
VERB: 71 73 81 85 86.
vhi: 73 74 87 88.
vlo: 8 54 74 76 77.
w: 16 17 18 19 20 21 23 24 29 30 31 41 75.
wall: 72 81 84.
wgt: 67.
work_t: 14 16 17 18 19 20 21 23 24 75.
wt: 36 42 43 44 70.
X: 64.
x: 18 35 67 81 82.
xrealloc: 37 38 40 42.
y: 67.

List of Refinements

⟨ Add the self term and the terms for the side and the top 66 ⟩ Used in chunk 61.
⟨ Append the results to the results file 89 ⟩ Used in chunk 84.
⟨ Count a single case and print the result 87 ⟩ Used in chunk 84.
⟨ Count all cases and add them up by type of centre 88 ⟩ Used in chunk 84.
⟨ Create the initial states 76 ⟩ Used in chunk 73.
⟨ Decide cell *ci* of the row for all states 78 ⟩ Used in chunk 73.
⟨ Determine the cells of row *k* 77 ⟩ Used in chunk 73.
⟨ Extend state *si* by the two choices for cell *i* 79 ⟩ Used in chunk 78.
⟨ Find the runs of the frontier 62 ⟩ Used in chunk 61.
⟨ Global variables 8 13 15 26 33 53 69 71 83 ⟩ Used in chunk 2.
⟨ Include files 3 ⟩ Used in chunk 2.
⟨ Initialise the height computation with the frontier cells 63 ⟩ Used in chunk 61.
⟨ Local variables of *count_case* 75 ⟩ Used in chunk 73.
⟨ Merge every state with its mirror image 82 ⟩ Used in chunk 73.
⟨ Parse the command line 85 ⟩ Used in chunk 84.
⟨ Pay for the outer paths, the rings and the middle 65 ⟩ Used in chunk 61.
⟨ Print the final statistics 91 ⟩ Used in chunk 84.
⟨ Print the program parameters 86 ⟩ Used in chunk 84.
⟨ Print the table of results 90 ⟩ Used in chunk 84.
⟨ Scan the runs from left to right and pay for the pockets 64 ⟩ Used in chunk 61.
⟨ Set up the geometry of the case 74 ⟩ Used in chunk 73.
⟨ Sharpen *lb* by looking one cell ahead 80 ⟩ Used in chunk 79.
⟨ Subroutines 9 16 17 18 19 20 21 23 24 29 30 31 34 35 37 38 39 40 41 42 43 44 46 54 55 56 57 58 59 60 61 67 68 70 72 73 ⟩ Used in chunk 2.
⟨ Swap the tables and update the statistics 81 ⟩ Used in chunk 78.
⟨ The main program 84 ⟩ Used in chunk 2.
⟨ Type definitions 14 22 27 32 36 ⟩ Used in chunk 2.